

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – PROMPT ENGINEERING CHALLENGE

Course Code:	CSA65	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO2 – Precision (BL2, BL3)	Assessment:	Assessment Tool 1 – Transformer Mechanics Worksheet
Weightage:	40% of CIA	Total Marks:	2 * 50 = 100 (2 Questions)
CO1 Statement:	<i>apply prompt engineering techniques and utilize pre-trained Large Language Models through modern AI frameworks and APIs to solve real-world problems (BL2, BL3)</i>		
Student Name:	Y. Jayesh Ranjan	Reg. No.:	192472201
Section / Batch:	CSE – AI	Date:	04/08/2026

Instructions to Students

- Answer 2 questions. Each question carries 50 marks. Total 100 Marks.
- 1. Design appropriate prompts for the given tasks using suitable prompt engineering techniques.
- 2. Select and apply the appropriate prompting strategy (Zero-shot, One-shot, Few-shot, Chain-of-Thought, or Structured Prompting) based on the given scenario.
- 3. Include clear instructions, relevant context, necessary constraints, and the expected output format while designing each prompt.
- 4. Evaluate and refine the designed prompts to improve their clarity, effectiveness, and quality by applying prompt engineering best practices.
- 5. Design prompts for a variety of real-world applications, such as text generation, summarization, translation, question answering, code generation, and conversational AI.
- 6. Justify the designed prompt by briefly explaining the selected prompting technique and how it helps achieve the desired output.

Code of Conduct

I Y. Jayesh Ranjan – 192472201 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Y. Jayesh Ranjan

aSECTION A – PROMPT ENGINEERING CHALLENGE

5. Create a Structured Prompt to generate a professional email requesting leave from work.

Prompt:

You are a professional business communication assistant.

Task:

Generate a professional leave request email.

Context:

An employee needs to request three days of leave from work due to a family function.

Instructions:

- Use a professional and respectful tone.
- Include an appropriate subject line.
- Address the manager formally.
- Clearly mention the reason for leave.
- Specify the leave dates as 15 September 2026 to 17 September 2026.
- Mention that pending work will be completed before the leave period.
- Request approval politely.
- End the email with a professional closing and employee name.

Constraints:

- Keep the email between 180 and 220 words.
- Do not use informal language or abbreviations.
- Ensure proper grammar and formatting.

Expected Output Format:

Subject:

Email Body:

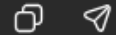
Closing:

Employee Name:

Output:

Here is a professionally formatted leave request email that meets all the specified requirements.

 Edit



Request for Leave from 15 September 2026 to 17 September 2026

Subject:

Request for Leave from 15 September 2026 to 17 September 2026

Email Body:

Dear Manager,

I hope this message finds you well. I am writing to formally request leave for three days, from **15 September 2026 to 17 September 2026**, as I need to attend an important family function that requires my presence throughout this period.

I understand the importance of maintaining continuity in my responsibilities and ensuring that my absence does not affect the team's workflow. To minimize any inconvenience, I will complete all pending assignments and prioritize any urgent tasks before the commencement of my leave. Additionally, I will ensure that all relevant updates and necessary information are shared with my colleagues so that work can continue smoothly during my absence.

I respectfully request your approval for this leave. Your consideration and support would be greatly appreciated. Please let me know if there are any additional formalities or arrangements that I should complete before my leave begins. I remain committed to fulfilling my responsibilities and will resume work promptly on my return.

Closing:

Yours sincerely,

Employee Name:

Rahul Sharma



26. Design a prompt that explains a programming error and suggests possible fixes.

Prompt:

You are an experienced software engineer and programming mentor.

Task:

Analyze the given programming error and explain it step by step.

Programming Error:

TypeError: unsupported operand type(s) for +: 'int' and 'str'

Instructions:

1. Explain what the error means in simple language.
2. Identify why this error occurs.
3. Describe the most likely causes.
4. Explain the debugging process step by step.
5. Suggest at least three possible solutions.
6. Provide corrected Python code for each solution.
7. Mention best practices to avoid this error in future programs.

Constraints:

- Explain in beginner-friendly language.
- Use clear headings.
- Keep the explanation concise but complete.
- Use Python examples only.

Expected Output Format:

1. Error Explanation

2. Cause of Error

3. Step-by-Step Debugging Process

4. Possible Fixes

5. Corrected Python Code

6. Best Practices

7. Conclusion

Output:

The image displays three screenshots of a ChatGPT interface, showing a step-by-step guide to debugging a Python error. The interface is dark-themed with a sidebar on the left and a main content area on the right.

1. Error Explanation

Error:

```
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

This error means that Python is trying to add an integer (`10`) and a string (`"10"`) using the `+` operator.

For example:

```
10 + "10"
```

Python cannot combine text (`str`) and numbers (`int`) directly, so it raises a `TypeError`.

2. Cause of Error

This error occurs because the `+` operator requires both values to be of the same data type.

Example:

```
number = 10
text = "10"
result = number + text
```

Error:

- `10` is an integer.
- `"10"` is a string.

Since Python does not automatically convert between these types, it throws the error.

Most Likely Causes

- Adding a number directly to a string.
- Use input from `input()` is always a string.
- Reading numeric values from a file without converting them.
- Mixing variables of different data types accidentally.

3. Step-by-Step Debugging Process

Step 1: Read the error message

Identify which data types Python is trying to combine.

```
"10" and "10"
```

This tells you an integer and a string are being added.

Step 2: Find the line causing the error

Look at the line number shown in the traceback.

Example:

```
1 result = number + text
```

Step 3: Check variable types

Use `type()` to inspect each variable.

```
1 print(type(number))
2 print(type(text))
```

Output:

```
1 <class 'int'>
2 <class 'str'>
```

Step 4: Decide the correct data type

Ask yourself:

- Should both values be numbers?
- Should both values be text?

Step 5: Convert the data

Use the appropriate conversion function before performing the operation.

4. Possible Fixes

Fix 1: Convert the string to an integer

Use this when the string contains a numeric value.

```
1 number = 10
2 text = "10"
3
4 result = number + int(text)
5 print(result)
```

Output:

```
1
2
3
4
5 20
```

ChatGPT

Fix 2: Convert the integer to a string
Use this when you want to display text.

```
Python
number = 30
text = "10"

result = int(number) + int(text)
print(result)
```

Output

```
Python
100
```

Fix 3: Use formatted strings (f-strings)
This is the recommended way to combine text and numbers.

```
Python
age = 30

message = f"Py Age: {age} Days!"
print(message)
```

Output

```
Python
Py Age: 30
```

ChatGPT

5. Corrected Python Code

Solution 1: Integer Addition

```
Python
number = 30
text = "10"

result = number + int(text)
print(result)
```

Solution 2: String Concatenation

```
Python
number = 30
text = "10"

result = str(number) + text
print(result)
```

Solution 3: Using f-Strings

```
Python
number = 30

print(f"The value is {number}")
```

Solution 4: Convert User Input

```
Python
first = input("Enter first number: ")
second = input("Enter second number: ")

result = int(first) + int(second)
print(result)
```

6. Best Practices

- Always check the data type of variables using `type()`.
- Remember that `input()` always returns a string.
- Convert user input to `int` or `float` before performing arithmetic.
- Use f-strings for combining text and numbers.
- Use meaningful variable names to make data types easier to understand.
- Validate user input before converting it to avoid invalid values.

7. Conclusion

The error `TypeError: unsupported operand type(s) for +: 'int' and 'str'` occurs because Python cannot directly add an integer and a string. To fix it, identify the variable types, determine whether you need numeric addition or text concatenation, and convert the values appropriately using `int()`, `str()`, or f-strings. Following good practices such as checking data types and validating user input will help prevent this error in future Python programs.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technique Selection	30%	All key concepts (AI, ML, DL, GenAI, LLMs) correctly identified with clear hierarchical relationships	Most concepts correct; minor gaps in relationships	Several concepts missing or misclassified	Major conceptual errors; map incomplete
Output Effectiveness	25%	Real-world examples linked accurately to each concept	Examples mostly relevant	Few or generic examples	No real-world linkage shown
Prompt Craftsmanship	20%	Logical, well-organized structure with clear connectors	Mostly organized; minor structural issues	Some disorganization affecting clarity	Poorly structured, hard to follow
Prompt-Output Documentation	15%	Visually clear, creative, easy to interpret	Neat and readable	Cluttered or inconsistent formatting	Untidy, difficult to interpret
Design Rationale	10%	Concise labels/notes that add clarity	Adequate labeling	Minimal or unclear labeling	No supporting labels or explanation

Marks Evaluation:

Question No.	Technical / Concept(30%)	Application(25%)	Quality (20%)	Presentation (15%)	Documentation / Communication (10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____